

SkyThermal Front-End Reproduction Guide

How to build a Windy/Ventusky-style weather-map front end for a paragliding meteo platform

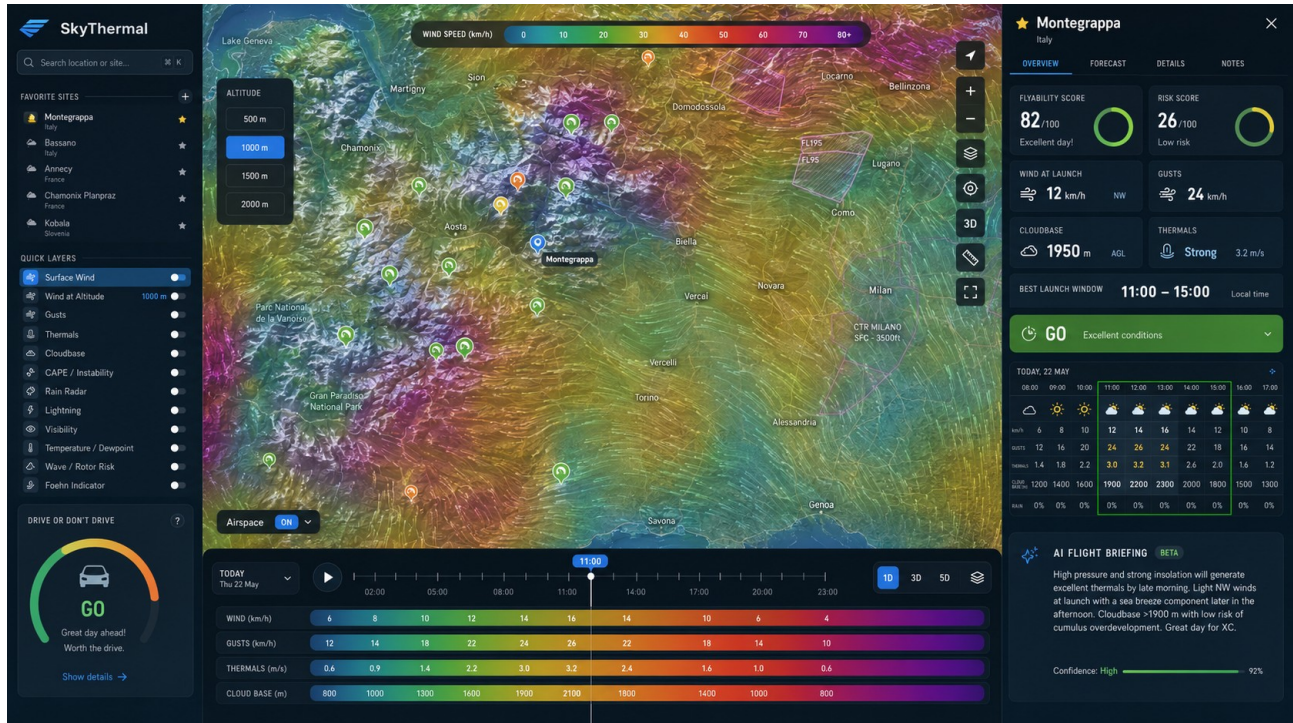


Figure 1 - SkyThermal target layout mockup: map-first paragliding weather dashboard.

Important legal/design boundary: this guide explains how to reproduce the front-end pattern and interaction architecture of modern weather-map applications, not how to clone Windy's brand, artwork, private code, protected assets, or exact visual identity. The final SkyThermal product must use its own brand, icons, color system, data pipeline, components, and user flows.

Table of Contents

1. 1. Goal and legal scope
2. 2. Windy-style UI decomposition
3. 3. Target SkyThermal layout
4. 4. Technology stack
5. 5. Application state model
6. 6. Weather layer registry
7. 7. Map rendering pipeline
8. 8. Animated wind particles
9. 9. Raster layers, isolines and legends
10. 10. Forecast timeline and player
11. 11. Weather picker
12. 12. Side panels and launch decision panel
13. 13. Paragliding-specific UX upgrades
14. 14. API and data contracts
15. 15. Performance, caching and offline strategy
16. 16. Mobile version
17. 17. Component architecture
18. 18. Styling and design tokens
19. 19. Testing and QA
20. 20. Implementation roadmap
21. Appendices

1. Goal and Legal Scope

The goal is to build a high-quality weather-map front end that feels as fluid and useful as Windy or Ventusky, but is specialized for paragliding decisions. The app should not be a generic weather viewer. It should answer: Can I fly? Should I drive? Which launch is best? What is the risk?

- Reproduce interaction patterns: full-screen map, layer menu, animated particles, forecast timeline, weather picker, site detail panel and forecast model/altitude controls.
- Do not reproduce protected elements: Windy logo, exact visual identity, proprietary icons, private code, exact color gradients, exact menu structure, or copied map tiles.
- Use open or licensed building blocks: MapLibre GL JS for map rendering, deck.gl or custom WebGL for overlays, your own tile/weather APIs, and your own product design language.
- Add paragliding-specific intelligence: launch-site fit, altitude wind, rotor/föhn warnings, thermals, cloudbase, airspace, landing fields, drive/no-drive decisions and pilot-level safety scoring.

Do	Do not
Use map-first layout and transparent control panels.	Copy Windy branding, CSS, icons or proprietary UI artwork.
Build your own weather layer registry and shader pipeline.	Scrape or reuse restricted tiles/data without license.
Use similar concepts: timeline, picker, particles, isolines.	Duplicate exact menus, text, layout proportions or trademarked names.
Differentiate with paragliding-specific decision intelligence.	Ship generic weather UI with no aviation safety logic.

2. Windy-Style UI Decomposition

Modern weather-map products are composed of five visible UI systems and one invisible state system. Reproducing the experience means building these systems, not just drawing a map.

Subsystem	Windy-style behavior	SkyThermal implementation
Map shell	Full-screen interactive weather map.	MapLibre terrain/vector map with weather overlays, launches, landings and airspace.
Layer control	Weather layers and model/level options.	Layer groups: Launch Safety, XC Potential, Danger, Navigation.
Particles and isolines	Animated flow plus contour overlays.	WebGL wind particles, pressure/cloudbase/thermal isolines, risk polygons.
Timeline/player	Scrub forecast time and animate changes.	Paragliding rows: wind, gusts, thermals, cloudbase, risk and flyability.
Picker	Click location and inspect point forecast.	Click map/site and show flyability, risk, launch fit and nearest landing.
State model	Overlay, level, model and time drive UI.	URL-backed state store for shareable forecasts and club briefing links.

Windy's public Map Forecast documentation describes weather visualizations around layers, particles and isolines, controlled by UI controls such as menus, progress bar, legend and picker. The SkyThermal architecture should implement the same category of primitives but with original product design and paragliding-specific semantics.

3. Target SkyThermal Desktop Layout



- Logo/search	- Terrain + weather overlay	- Selected launch	
- Favorite sites	- Wind particles	- Flyability/risk	
- Quick layers	- Launch/landing markers	- Best launch window	
- Drive/no-drive	- Altitude selector	- Mini forecast table	
	- Airspace/rotor zones	- AI briefing	
Bottom Forecast Timeline: play button, time scrubber, wind/gust/thermal/cloudbase/risk rows			

- The map must occupy the majority of the screen. Controls float above it rather than replacing it.
- The left panel is for selection and filters: search, favorites, quick layers and drive decision.
- The right panel is for decision detail: site score, risk, launch window, hourly table and AI briefing.
- The bottom timeline controls forecast time and shows the most important paragliding variables across the day.

4. Recommended Technology Stack

Layer	Recommended tool	Reason
Framework	React + TypeScript + Vite or Next.js	Fast development, component ecosystem, SSR option for marketing/portal pages.
Map engine	MapLibre GL JS	Open-source WebGL vector map rendering, style-driven maps and custom layers.
Advanced overlays	deck.gl	GPU-oriented visualization layers, good for large datasets and custom overlays.
Particles	Custom WebGL layer or deck.gl particle layer	Needed for smooth animated wind/thermal flow.
State	Zustand or Redux Toolkit	Centralized map state and URL synchronization.
Server cache	TanStack Query	Forecast and site data caching with refetch/invalidation.
UI components	Custom components + Radix UI primitives	Accessibility without locking into a generic dashboard look.
Styling	CSS variables + Tailwind optional	Design token system and fast iteration.
Testing	Playwright + Vitest + Storybook	Visual regression, component states and end-to-end flows.

MapLibre GL JS is a TypeScript/WebGL library for rendering interactive vector-tile maps in the browser. deck.gl is designed for high-performance GPU/WebGL or WebGPU visual analytics. These two tools are a strong open foundation for a Windy-style weather-map application.

5. Application State Model

The front end must be state-driven. A Windy-style UI feels coherent because every visible control updates the same underlying state: overlay, altitude/level, forecast model, timestamp, selected position and selected site.

```
type SkyThermalMapState = {
  overlay:
    | 'surface_wind'
```

```

| 'altitude_wind'
| 'gusts'
| 'thermals'
| 'cloudbase'
| 'cape_instability'
| 'rain_radar'
| 'lightning'
| 'visibility'
| 'temperature_dewpoint'
| 'pressure_trend'
| 'wave_rotor_risk'
| 'foehn_indicator';

altitudeM: 0 | 500 | 1000 | 1500 | 2000 | 3000;
forecastModel: 'auto_best' | 'ecmwf' | 'gfs' | 'met_norway' | 'icon' | 'arome';
validTime: string;    // ISO timestamp
selectedSiteId?: string;
selectedPoint?: { lat: number; lon: number };
airspaceEnabled: boolean;
particlesEnabled: boolean;
isolinesEnabled: boolean;
timelinePlaying: boolean;
map: { center: [number, number]; zoom: number; bearing: number; pitch: number };
};

```

Persist the state in the URL. This lets pilots, clubs and schools share exact forecast views.

```

/app?overlay=altitude_wind&altitude=1000&model=ecmwf&time=2026-06-05T12:00Z&site=montegrappa

```

6. Weather Layer Registry

A layer registry is the front-end contract that tells the UI how to render each weather layer. It prevents one-off layer logic from spreading through the app.

Layer	Type	Inputs	Renderer	Paragliding purpose
Surface wind	vector	u10, v10	particles + color raster	Launch safety.
Wind at altitude	vector	u/v at 500/1000/1500/2000m	particles + altitude selector	Drift and XC planning.
Gusts	scalar	gust_kmh	raster	Collapse/launch danger.
Wind direction	vector/angle	direction_deg	arrows/particles	Launch compatibility.
Thermals	scalar	thermal_strength_ms	raster + trigger markers	Soaring potential.
Cloudbase	scalar	cloudbase_m_agl	raster + isolines	XC potential/cloud risk.
CAPE/instability	scalar	cape_jkg, lifted_index	raster	Overdevelopment risk.
Rain radar	raster/nowcast	radar dbz/mmh tiles	animated raster tiles	No-fly zones.
Lightning	point/time	strike location/time	points + age fade	Critical danger.
Visibility	scalar	visibility_km	raster	Safe navigation.
Temp/dewpoint	scalar pair	temp_c, dewpoint_c	raster + labels	Cloud formation.
Pressure trend	scalar/vector	mslp, trend_hpa_3h	isolines + arrows	Weather change.

Wave/rotor risk	derived scalar	terrain + wind + stability	risk polygons	Mountain danger.
Föhn indicator	derived scalar	pressure gradient + humidity + wind	risk overlay	Mountain weather danger.

```
export const layerRegistry = {
  altitude_wind: {
    label: 'Wind at Altitude',
    group: 'Launch Safety',
    variableType: 'vector',
    units: 'km/h',
    altitudeSelector: true,
    renderer: ['colorRaster', 'particles'],
    api: '/v1/map/layers/altitude-wind',
    entitlement: 'free',
    defaultOpacity: 0.78,
    colorScale: 'windSpeedKmh',
    legend: { min: 0, max: 80, ticks: [0,10,20,30,40,50,60,70,80] }
  },
  wave_rotor_risk: {
    label: 'Wave / Rotor Risk',
    group: 'Danger',
    variableType: 'derived_scalar',
    units: 'risk',
    renderer: ['riskPolygons', 'transparentRaster'],
    api: '/v1/map/layers/rotor-risk',
    entitlement: 'pro_plus',
    defaultOpacity: 0.55
  }
};
```

7. Map Rendering Pipeline

22. Initialize the base map with MapLibre using a dark terrain-friendly style.
23. Load base vector tiles: roads, water, boundaries, labels and terrain hillshade.
24. Add optional 3D terrain or hillshade for mountain readability.
25. Add weather raster/vector layers above the base map but below site markers.
26. Add paragliding objects: launches, landings, airspace, hazard zones and local playbook polygons.
27. Add UI overlays: legend, altitude selector, timeline, map controls and picker popup.
28. Update weather layers when overlay, altitude, model or timestamp changes.
29. Keep expensive particle animation in a custom WebGL layer or worker-assisted deck.gl layer.

```
<MapShell>
<MapLibreMap styleUrl="/styles/skythermal-dark-terrain.json" />
<WeatherRasterLayer overlay={state.overlay} time={state.validTime} />
<WindParticleLayer overlay={state.overlay} altitudeM={state.altitudeM} />
<AirspaceLayer enabled={state.airspaceEnabled} />
<LaunchMarkerLayer />
<LandingMarkerLayer />
<HazardZoneLayer />
```

```

<WeatherPicker />
<MapControls />
</MapShell>

```

8. Animated Wind Particles

Windy-style motion depends on particles advected through a vector field. The vector field is usually represented by U/V wind components. The renderer samples U/V at each particle position, moves the particle, fades trails and recolors by speed.

```

// Simplified particle step
for each particle:
  uv = sampleVectorTexture(particle.lon, particle.lat, altitudeM, validTime)
  speed = length(uv)
  particle.lon += uv.u * dt * scale
  particle.lat += uv.v * dt * scale
  particle.age += dt
  color = colorRamp(speed)
  drawTrail(particle.previousPosition, particle.position, color, alpha)
if particle.age > maxAge or particle leaves viewport:
  respawn particle randomly inside viewport

```

Parameter	Recommended default	Notes
Particle count desktop	8,000-30,000	Adapt to GPU, zoom and viewport size.
Particle count mobile	1,500-6,000	Battery and thermals matter.
Frame target	30-60 fps	30 fps acceptable on mobile.
Trail fade	0.88-0.96	Longer trails show flow, shorter trails improve clarity.
Color source	wind speed	Use same legend as raster layer.
Data texture	RGBA float or encoded U/V	Use typed arrays or GPU textures.

9. Raster Layers, Isolines and Legends

A complete weather-map UI needs three overlay types: raster heatmaps, isolines and point/geometry overlays. Each layer should share a legend, opacity, unit and data-provenance model.

Overlay type	Use for	Implementation
Raster heatmap	Wind speed, gusts, thermals, cloudbase, CAPE, visibility	Serve PNG/WebP tiles or GPU-decoded data tiles.
Vector field	Wind, wave, convergence	Serve U/V tiles and animate particles.
Isolines	Pressure, cloudbase, freezing level	Precompute contours server-side or generate client-side for small extents.
Points	Lightning, stations, webcams, pilot reports	Cluster at low zoom, show precise markers at high zoom.
Polygons	Airspace, rotor zones, no-fly zones, danger areas	Vector tiles or GeoJSON depending on scale.

```

type WeatherTileDescriptor = {
  layerId: string;
  model: string;
  runTime: string;
}

```



```

validTime: string;
altitudeM?: number;
urlTemplate: string; // /tiles/weather/{layer}/{z}/{x}/{y}.webp?...
units: string;
min: number;
max: number;
colorScale: string;
attribution: string[];
};

```

10. Forecast Timeline and Animation Player

The bottom timeline is the control surface for time. It should not only show hours; it should show why a time is safe or unsafe for paragliding.

Row	Data source	UI behavior
Wind	site_forecast.wind_kmh	Color bar with hourly values.
Gusts	site_forecast.gust_kmh	Highlight when gust threshold exceeded.
Thermals	derived_features.thermal_strength_ms	Highlight optimal XC window.
Cloudbase	derived_features.cloudbase_m_agl	Warn when below terrain margin.
Rain	radar/forecast precip	Red no-fly blocks.
Risk	risk_engine.risk_score	NO-GO shading over unsafe intervals.
Flyability	flyability_score	Green/yellow/red launch window.

```

function onTimelineScrub(validTime: string) {
  mapState.set({ validTime, timelinePlaying: false });
  queryClient.invalidateQueries(['weather-layer', overlay, altitudeM, validTime]);
  queryClient.invalidateQueries(['site-forecast', selectedSiteId, validTime]);
}

```

```

function playForecast() {
  every(650ms, 0 => mapState.advanceToNextForecastStep());
}

```

11. Weather Picker

The picker should be upgraded beyond generic point weather. It should connect the clicked point to terrain, nearest launch, nearest landing, airspace and paragliding risk.

```
GET /v1/picker?lat=45.843&lon=11.778&time=2026-06-05T12:00Z&model=auto_best&altitude_m=1000
```

Response:

```

{
  "point": { "lat": 45.843, "lon": 11.778, "terrain_altitude_m": 1040 },
  "weather": {
    "wind_kmh": 18,
    "wind_direction_deg": 315,
    "gust_kmh": 26,

```



```

"thermal_strength_ms": 2.7,
"cloudbase_m_agl": 1850,
"rain_probability": 0,
"cape_jkg": 80
},
"paragliding": {
  "nearest_launch_id": "montegrappa-main",
  "nearest_landing_id": "bassano-landing",
  "airspace_status": "clear",
  "rotor_risk": "moderate",
  "recommendation": "MAYBE"
}
}

```

12. Side Panels and Launch Decision Panel

Panel	Purpose	Key components
Left sidebar	Navigation and filters	Search, favorites, layer groups, drive/no-drive card.
Right site panel	Decision detail	Flyability score, risk score, wind, gusts, cloudbase, thermals, launch window, AI briefing.
Bottom timeline	Time control	Play/scrub, layer rows, best window highlight.
Floating controls	Map tools	Zoom, locate, layers, 3D, measure, fullscreen.
Mobile bottom sheet	Small-screen site detail	GO/MAYBE/NO-GO, swipeable tabs, compact hourly chart.

```

type SiteDecision = {
  siteId: string;
  validTime: string;
  flyabilityScore: number;
  riskScore: number;
  recommendation: 'GO' | 'MAYBE' | 'NO_GO' | 'NO_DATA';
  bestWindow: { start: string; end: string; confidence: number };
  blockers: Array<{ code: string; severity: 'warning' | 'danger'; message: string }>;
  explanation: string[];
};

```

13. Paragliding-Specific UX Upgrades

- Drive or Don't Drive: ranks reachable sites based on drive time, best launch window, confidence and live reports.
- Pilot level personalization: beginner, intermediate, advanced, tandem and school-student thresholds.
- Site playbook integration: each launch has local danger winds, rotor zones, landing notes and club rules.
- Forecast confidence: show model disagreement and live station confirmation before recommending GO.
- Live reality layer: pilot reports, station observations, webcams and local expert validation.
- In-flight mode: simplified map, audio alerts, glide-to-landing, airspace warnings and emergency sharing.

14. API and Data Contracts

Endpoint	Used by	Returns
GET /v1/map/layers	Layer panel	Available layers, entitlement and legends.
GET /v1/map/layers/{layer}/tiles/{z}/{x}/{y}	Map renderer	Raster/vector weather tile.
GET /v1/map/wind-vector-tile	Particle renderer	U/V data tile for time/model/altitude.
GET /v1/sites/search	Search box	Launch/site results.
GET /v1/sites/{id}/forecast	Right panel and timeline	Hourly site forecast and scores.
GET /v1/sites/{id}/decision	Decision panel	GO/MAYBE/NO-GO, blockers and confidence.
GET /v1/picker	Map click picker	Point weather + paragliding context.
GET /v1/airspace/tiles/{z}/{x}/{y}	Airspace overlay	Vector tile or GeoJSON.
POST /v1/alerts/rules	Alert builder	Saved alert condition.

15. Performance, Caching and Offline Strategy

- Use tile-based rendering for all large spatial datasets. Never send full forecast grids to the browser.
- Cache layer metadata, legends and site basics aggressively. Forecast tiles can use short TTLs tied to model-run updates.
- Use Web Workers for decoding GRIB-derived vector tiles or heavy interpolation.
- Throttle particle updates when the map is moving or tab is hidden.
- Use lower particle density on mobile and battery saver mode.
- Preload the next two forecast times during timeline playback.
- Persist favorite sites, recent tiles, selected area and emergency map data for offline use.

Metric	Target
Initial app shell load	< 2.5 s on good 4G desktop/laptop.
Map first render	< 2.0 s after app shell.
Layer switch	< 700 ms cached, < 2 s uncached.
Timeline scrub response	< 300 ms for UI state, layer loads async.
Particle FPS desktop	30-60 fps.
Particle FPS mobile	24-30 fps acceptable.
Picker response	< 800 ms p95.

16. Mobile Version

Mobile layout:

Full-screen map

- └ Top compact search/location chip
- └ Floating layer button
- └ Floating timeline button
- └ Emergency/SOS access
- └ Bottom sheet
 - └ collapsed: GO/MAYBE/NO-GO + site name
 - └ half: key metrics + launch window
 - └ full: hourly table + AI briefing + details

- Do not copy desktop sidebars on mobile. Use bottom sheets and floating action buttons.
- In-flight mode must reduce visual complexity and support audio/haptic warnings.

- Offline mode should keep maps, launch playbook, airspace cache and last forecast for favorite sites.
- Layer toggles should be grouped into compact presets: Launch Safety, XC Potential, Danger, Navigation.

17. Component Architecture

```
src/  
  app/  
    AppShell.tsx  
    routes.tsx  
  state/  
    mapState.ts  
    entitlementState.ts  
    timelineState.ts  
  map/  
    MapShell.tsx  
    MapLibreMap.tsx  
    WeatherRasterLayer.tsx  
    WindParticleLayer.tsx  
    AirspaceLayer.tsx  
    LaunchMarkerLayer.tsx  
    WeatherPicker.tsx  
  panels/  
    LeftSidebar.tsx  
    LayerPanel.tsx  
    SiteDecisionPanel.tsx  
    ForecastTimeline.tsx  
    AIFlightBriefingCard.tsx  
    DriveDecisionCard.tsx  
  api/  
    weatherApi.ts  
    sitesApi.ts  
    pickerApi.ts  
    alertsApi.ts  
  design/  
    tokens.css  
    layer-colors.ts  
    safety-colors.ts
```

Component	Owns	Does not own
MapShell	Map container, overlay composition.	Business scoring.
LayerPanel	Layer selection, entitlement display.	Weather data fetching internals.
ForecastTimeline	Time state, playback UI.	Forecast calculations.
SiteDecisionPanel	Decision display and explanations.	Risk algorithm implementation.
WindParticleLayer	GPU rendering of vector field.	Layer registry/business naming.
WeatherPicker	Point inspection UI.	Terrain/weather computation.

18. Styling and Design Tokens

```
:root {  
  --bg-app: #06101d;  
  --panel-bg: rgba(7, 18, 31, 0.88);
```

```

--panel-border: rgba(170, 216, 255, 0.13);
--text-main: #edf7ff;
--text-muted: #8fa8bc;

--decision-go: #43b547;
--decision-maybe: #f2b632;
--decision-no-go: #e84d5b;
--decision-no-data: #7b8794;

--wind-0: #267bd4;
--wind-10: #1cc0b2;
--wind-20: #59c55a;
--wind-30: #d2bd32;
--wind-40: #f1852d;
--wind-50: #dd484a;
--wind-60: #c33190;
--wind-80: #732bce;
}

```

- Use dark translucent panels so the map remains visible.
- Use safety colors consistently: green means GO, yellow means MAYBE, red means NO-GO, grey means NO DATA.
- Use units in every metric. Wind without units is unsafe UX.
- Use icons as support only; never rely on color alone for safety decisions.
- Use smooth transitions for panels, but avoid decorative animation during in-flight mode.

19. Testing and QA

Test type	What to verify
Unit tests	Layer registry, state reducers, URL serializer, entitlement gates.
Component tests	Layer panel, timeline, site panel, picker popup, GO/MAYBE/NO-GO states.
E2E tests	Select layer, change altitude, scrub time, click site, view decision, create alert.
Visual regression	Desktop and mobile screenshots for main map states.
Performance tests	Layer switch speed, particles FPS, timeline playback stability.
Safety UX tests	NO-GO blockers always visible, AI never hides uncertainty.
Accessibility tests	Keyboard navigation, contrast, focus states, screen-reader labels.

```

// Example Playwright flow
await page.goto('/app?site=montegrappa&overlay=surface_wind');
await page.getByRole('button', { name: 'Wind at Altitude' }).click();
await page.getByRole('button', { name: '1000 m' }).click();
await page.getByText('Flyability Score').isVisible();
await expect(page.getByText(/GO | MAYBE | NO-GO/)).toBeVisible();

```

20. Implementation Roadmap

Phase	Duration	Deliverables
Phase 0 - Prototype	1-2 weeks	Static React layout, fake map, fake layers, mock site panel, design tokens.
Phase 1 - Real map shell	2-4 weeks	MapLibre map, marker layers, airspace overlay, URL state, sidebars.
Phase 2 - Weather tiles	4-8 weeks	Weather layer API integration, raster tiles, legend, timeline, picker.
Phase 3 - Particles	4-8 weeks	U/V vector tile pipeline, WebGL/deck.gl particle layer, altitude selector.
Phase 4 - Paragliding intelligence	6-10 weeks	Flyability, risk, site playbooks, drive/no-drive, AI briefing.
Phase 5 - Mobile/offline	6-10 weeks	Bottom-sheet mobile UI, offline cache, in-flight mode basics.
Phase 6 - Premium polish	ongoing	Model comparison, forecast confidence, advanced layers, paid entitlements.

Appendix A - Minimum Viable Front-End Checklist

- Map renders with terrain/dark style.
- Layer panel shows all core paragliding layers.
- Altitude selector updates altitude wind request.
- Timeline scrubber changes valid time and reloads layer/site forecast.
- Launch markers are clickable.
- Right panel shows flyability, risk, wind, gusts, cloudbase, thermals and launch window.
- Picker returns point forecast and nearest launch.
- URL contains selected layer, time, model, altitude and site.
- GO/MAYBE/NO-GO colors and language are consistent.
- No proprietary Windy assets, names or copied layout details are used.

Appendix B - Reference Sources

- **Windy Map Forecast API documentation:** <https://api.windy.com/map-forecast/docs>
- **Windy weather picker tutorial:** <https://api.windy.com/map-forecast/tutorials/picker>
- **Windy API overview:** <https://api.windy.com/>
- **MapLibre GL JS documentation:** <https://www.maplibre.org/maplibre-gl-js/docs/>
- **deck.gl documentation:** <https://deck.gl/docs>
- **ECMWF Open Data:** <https://www.ecmwf.int/en/forecasts/datasets/open-data>
- **ECMWF real-time open data GRIB2 availability:**
<https://www.ecmwf.int/en/forecasts/access-forecasts/access-real-time-open-data>
- **MapTiler weather layer examples:** <https://docs.maptiler.com/sdk-js/examples/weather-pressure/>